

Triton Programming — Placement and Interview Guide

Triton is a Python-based language and compiler for writing custom GPU kernels. This guide assumes basic Python, tensors, and GPU vocabulary. Examples use `triton` and `triton.language as tl`; they are intentionally small enough to discuss in an interview while exposing the performance ideas interviewers care about: work decomposition, memory access, reductions, tiling, numerical stability, and fusion.

Version note: Triton evolves quickly. Exact tuning values and some advanced APIs are hardware- and release-dependent. The programming-model ideas in this guide are the stable part; benchmark on the target GPU before claiming a configuration is optimal.

Contents

1. [Triton Programming Model](#)
2. [Program IDs](#)
3. [Blocks](#)
4. [Masked Loads and Stores](#)
5. [Vector Addition](#)
6. [Softmax](#)
7. [Matrix Multiplication](#)
8. [LayerNorm](#)
9. [Fused Kernels](#)

Triton Programming Model

1. Overview

Definition. Triton is a domain-specific language embedded in Python for writing GPU kernels. A function decorated with `@triton.jit` describes one **program instance** operating on a block of tensor elements. A launch grid creates many instances of that program.

Triton matters because it exposes more control than a tensor framework while hiding much of CUDA's thread-by-thread machinery. It is used for custom deep-learning operators, research kernels, compiler-generated operators, and fused replacements for sequences of PyTorch operations. Interviewers ask about it to test whether you can map tensor computation to parallel blocks, reason about memory traffic, and distinguish compilation-time choices from runtime values.

2. Core Idea

The central abstraction is **blocked SPMD**: Single Program, Multiple Data. Imagine dividing a long spreadsheet into chunks and giving each worker the same formula plus a chunk number. The formula is the Triton kernel; the chunk number is `tl.program_id`; the chunk is a vector of indices created with `tl.arange`.

```

import triton
import triton.language as tl

@triton.jit
def square_kernel(x_ptr, y_ptr, n, BLOCK: tl.constexpr):
    pid = tl.program_id(0)
    offsets = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n
    x = tl.load(x_ptr + offsets, mask=mask, other=0.0)
    tl.store(y_ptr + offsets, x * x, mask=mask)

# square_kernel[(triton.cdiv(n, 256),)](x, y, n=n, BLOCK=256)

```

Step by step: Python launches a grid; each program obtains its ID; vector offsets describe a block of elements; masked loads bring valid elements into values the compiler can keep in registers/on-chip storage; arithmetic is expressed over the entire block; masked stores write results. Triton's compiler lowers this blocked description to hardware threads, warps/wavefronts, memory operations, and synchronization.

3. Important Subtopics

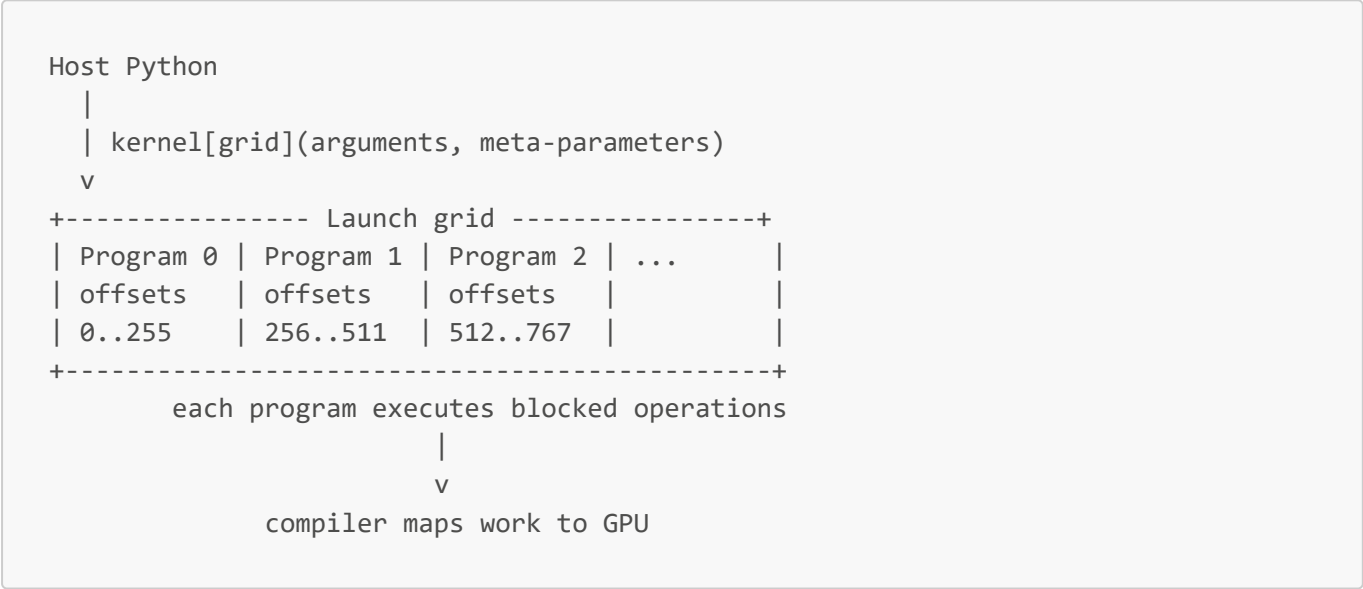
Subtopic	What it means and why it matters	Example	Common interview angle
JIT kernel	<code>@triton.jit</code> compiles a restricted Python-like function for the GPU.	<code>@triton.jit def add_kernel(...)</code>	Host Python is not executed once per GPU element.
Program instance	One logical kernel instance handles a block, not one scalar.	One program processes 256 elements.	Contrast with CUDA's usual one-thread mental model.
Launch grid	Tuple giving the number of program instances on each axis.	<code>(ceil(n/BLOCK),)</code> or <code>(M, ceil(N/BLOCK))</code>	Grid size and block shape are separate ideas.
Blocked values	<code>tl.arange</code> and broadcasting create vectors/matrices of values.	<code>rows[:, None], cols[None, :]</code>	Explain vectorized pointer arithmetic.
<code>tl.constexpr</code>	A compile-time meta-parameter used for shapes, unrolling, and specialization.	<code>BLOCK: tl.constexpr</code>	It can improve code generation but causes variants to compile.
Pointer arithmetic	Tensor addresses are base pointer plus index/stride expressions.	<code>x + row*stride + col</code>	Correctness for non-contiguous tensors.
Execution tuning	<code>num_warps</code> , <code>num_stages</code> , tile sizes, and autotuning affect resource use.	<code>kernel[grid](..., num_warps=4)</code>	More warps or larger tiles are not automatically faster.

Subtopic	What it means and why it matters	Example	Common interview angle
Compilation pipeline	Triton IR is optimized and lowered to target GPU code.	Kernel specializes for constants and dtypes.	Triton is a compiler/language, not just a Python wrapper.

4. Real-World Example

In a transformer inference server, a model may need **bias + GELU** after a matrix multiplication. Calling two framework operators can launch two kernels and materialize an intermediate tensor. A Triton program can load the matmul output and bias, compute GELU, and store once. This reduces launch count and global-memory traffic while remaining callable as a normal Python operation.

5. Diagrams / Mental Models



Layer	Programmer normally controls	Compiler/runtime normally handles
Algorithm	Tiling, program-to-data mapping, fusion	Instruction selection
Memory	Pointer expressions, masks, reuse strategy	Many low-level transactions
Parallelism	Grid and block shapes	Mapping blocked work to lanes/warps
Tuning	Meta-parameters, candidate configurations	Compilation and launch

6. Common Interview Questions

Question	Clear answer	Key points expected	Common mistake
1. What is Triton?	A Python-embedded language and compiler for custom GPU kernels using blocked tensor programs.	Language, compiler, GPU kernels, blocked model.	Calling it a replacement for Python or all of CUDA.

Question	Clear answer	Key points expected	Common mistake
2. How is Triton's model different from CUDA?	Triton code describes block-level vector operations; CUDA commonly exposes individual threads, blocks, shared memory, and barriers explicitly.	Different abstraction level; both execute on GPUs.	Saying Triton has no threads underneath.
3. What does <code>@triton.jit</code> do?	It marks a function for specialization and compilation into a GPU kernel.	Restricted kernel language, JIT variants.	Treating arbitrary Python as valid device code.
4. What is the grid?	The logical shape/count of independent program instances.	One-, two-, or three-dimensional IDs.	Confusing it with <code>BLOCK_SIZE</code> .
5. What is a blocked program?	One instance operates on vector/tile-shaped data represented by Triton values.	Vector offsets and tensor tiles.	Assuming one instance equals one element.
6. Why use <code>tl.constexpr</code> ?	It makes a value known during compilation, enabling static shapes, specialization, and unrolling.	Compile-time versus runtime.	Marking every argument <code>constexpr</code> and causing excessive variants.
7. Where are intermediate values stored?	The compiler generally maps them to registers and may use on-chip memory/spills as required.	Resource pressure matters.	Promising that every value stays in SRAM/registers.
8. Does Triton remove the need to understand GPU hardware?	No. It removes boilerplate, but tiling, coalescing, occupancy, and numerical behavior still determine performance.	Abstraction is not automatic optimality.	Claiming the compiler fixes a poor algorithm.
9. When should you use Triton?	For performance-critical custom/fused tensor kernels not adequately served by optimized library operators.	Profile first; libraries often win for standard ops.	Rewriting every PyTorch operation.
10. How do you validate a kernel?	Compare against a trusted reference across shapes/dtypes, include edge cases, use tolerances, then benchmark after warm-up and synchronization.	Correctness before speed.	Testing one divisible shape or timing asynchronous launches.

7. Deep-Dive Questions

1. **How does specialization trade speed for cost?** Compile-time constants let the compiler remove branches and generate shape-specific code, but each distinct signature/meta-configuration can incur compilation time and cache pressure.

2. **What limits block size?** Register use, shared/on-chip memory, legal tensor shapes, warp allocation, and the work needed per program. An oversized tile can spill registers or reduce occupancy.
3. **How are multidimensional blocks formed?** Build independent ranges and broadcast them: `rows[:, None]` plus `cols[None, :]` creates a matrix of indices or pointers.
4. **What does the host wrapper own?** Shape/dtype validation, allocation, grid calculation, meta-parameter selection, dispatch, autograd integration, and a fallback if the kernel's supported domain is narrower.
5. **Why can a readable Triton kernel compete with handwritten CUDA?** The blocked IR exposes data layout, vectorization, and reuse to an optimizing compiler while avoiding some low-level scheduling code; it is not a guarantee, and vendor libraries may still be superior.

8. Comparison Tables

Aspect	Triton	CUDA C++	PyTorch tensor operation
Main abstraction	Blocked program	Threads, warps, blocks	Whole tensor/operator
Control	Medium-high	Highest	Low
Development effort	Moderate	High	Low
Fusion/customization	Strong	Strong	Depends on compiler/operator set
Portability	Supported Triton backends	Vendor ecosystem	Framework backends
Best fit	Custom DL/HPC tensor kernels	Maximum control/general GPU work	Standard model/application code
Runtime parameter		Compile-time meta-parameter	
Can vary without intentionally specializing algorithmic shape		Known to compiler and usable in static shapes	
Example: pointer, <code>n_elements</code> , stride		Example: <code>BLOCK_SIZE</code> , activation choice	
Used in normal arithmetic/control supported at runtime		Enables constant folding/unrolling	

9. Common Mistakes

- Thinking a Triton program is one GPU thread rather than one blocked instance.
- Launching `n` programs when each program already processes `BLOCK` elements.
- Assuming contiguous storage while ignoring strides.
- Making dynamic block shapes that must actually be compile-time constants.
- Benchmarking compilation time or unsynchronized asynchronous launches.
- Assuming bigger blocks, more warps, or higher occupancy always means better performance.
- Replacing highly optimized library kernels without profiling.

10. Edge Cases / Special Cases

- Empty tensors should usually be handled by the host wrapper to avoid a zero-sized/invalid launch.

- Non-power-of-two dimensions often require padded block sizes and masks.
- Non-contiguous tensors need explicit stride-aware address calculations or a documented contiguous-only contract.
- Different dtypes can change accumulation precision, register usage, and available hardware instructions.
- Very small inputs may be dominated by launch overhead; a fused parent operation may be the real optimization.
- Alias safety is kernel-specific; overlapping input/output can create cross-program races.

11. How to Explain in Interview

"Triton is a Python-embedded GPU kernel language based on blocked SPMD. I define how one program instance processes a vector or tile, launch a grid of those programs, and use program IDs, pointer arithmetic, and masks to cover the tensor safely. The compiler maps those block operations to GPU execution, while I still choose tiling and memory-access patterns that control performance."

12. Quick Revision Notes

- **Program:** one blocked kernel instance.
- **Grid:** number and shape of program instances.
- **Block:** vector/tile of elements processed by one program.
- **tl.constexpr:** value known during compilation.
- **Key performance levers:** coalescing, reuse, tile size, fusion, `num_warps`, `num_stages`.
- **Trap:** Triton simplifies GPU programming; it does not make hardware reasoning optional.

13. Practice Tasks

1. Implement square, ReLU, and affine-transform kernels with arbitrary lengths.
2. Change a flat kernel to accept 2D strides and verify a transposed input.
3. Print or reason through offsets for `n=10`, `BLOCK=8`, and two programs.
4. Benchmark block sizes 64–1024 and explain the curve rather than selecting the largest.
5. Compare compilation latency, first-call latency, and steady-state kernel latency.
6. Explain which adjacent PyTorch elementwise operations are safe and useful to fuse.

14. Final Cheat Sheet

Item	Answer
Core definition	Python-embedded compiler language for blocked GPU programs
Why it matters	Custom performance with less low-level code than CUDA
Most asked	Grid vs block, <code>program_id</code> , masks, <code>constexpr</code> , validation
Main comparison	Triton exposes tiles; CUDA exposes threads; PyTorch exposes operators
One-line answer	"Triton lets me map tensor tiles to GPU programs and compile them into efficient custom kernels."

Program IDs

1. Overview

Definition. `tl.program_id(axis)` returns the coordinate of the current Triton program instance along launch-grid axis 0, 1, or 2. `tl.num_programs(axis)` returns the number of programs launched on that axis.

Program IDs matter because all program instances run the same code; the ID tells each instance which disjoint data to own. They are used for vectors, rows, matrix tiles, batches, heads, and grouped workloads. Interviewers use them to test whether you can derive a correct data mapping, not merely recall syntax.

2. Core Idea

Think of numbered tickets at a service counter. Every worker follows the same procedure, but ticket 0 handles customers 0–127, ticket 1 handles 128–255, and so on.

```
pid = tl.program_id(axis=0)
offsets = pid * BLOCK + tl.arange(0, BLOCK)
```

For `BLOCK=4`, program 0 gets `[0,1,2,3]`, program 1 gets `[4,5,6,7]`, and program 2 gets `[8,9,10,11]`. A 2D grid can map `pid_m = tl.program_id(0)` to row tiles and `pid_n = tl.program_id(1)` to column tiles. Alternatively, a flat ID can be decoded using division and remainder.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Common interview angle
Axis	Selects a dimension of a 1D/2D/3D launch grid.	<code>program_id(1)</code> for column tile.	Axis must match launch-grid layout.
Linear mapping	One ID maps to one consecutive chunk.	<code>start=pid*BLOCK</code> .	Easiest route to coalesced vector access.
2D mapping	Separate IDs select independent tile dimensions.	Grid <code>(ceil(M/BM), ceil(N/BN))</code> .	Clear mapping versus possible scheduling/cache effects.
Flatten/decode	One linear ID represents a multidimensional coordinate.	<code>row=pid//tiles_n</code> , <code>col=pid%tiles_n</code> .	Avoid swapped dimensions and wrong divisor.
Grouped ordering	Reorders tile IDs to improve reuse/cache locality.	Process several M tiles for nearby N tiles.	Mapping changes order, not mathematical ownership.

Subtopic	Meaning and importance	Example	Common interview angle
Grid-stride loop	A program processes multiple tasks spaced by total program count.	<code>for row in tl.range(pid, M, tl.num_programs(0)).</code>	Persistent/fixed-size grids and load balancing.
Uniqueness	IDs are unique coordinates within the launch grid, but address expressions must remain unique for race-free writes.	Each <code>(pid_m, pid_n)</code> owns one C tile.	Unique ID does not guarantee unique memory.

4. Real-World Example

For batched attention with shape `[batch, heads, sequence, dimension]`, a grid can use one axis for query-row tiles and another flattened axis for `batch * heads`. The kernel decodes `off_bh` as `batch = off_bh // heads, head = off_bh % heads`, then computes the corresponding attention tile. This mapping exposes independent work without storing a task list.

5. Diagrams / Mental Models

2D grid (axis 0 = M tile, axis 1 = N tile)

		pid_n		
		0	1	2
pid_m	0	C00	C01	C02
	1	C10	C11	C12

Flattened pid for 3 N-tiles:

pid: 0 1 2 3 4 5
(m,n): (0,0)(0,1)(0,2)(1,0)(1,1)(1,2)
m = pid // 3, n = pid % 3

6. Common Interview Questions

Question	Clear answer	Key points expected	Common mistake
1. What does <code>tl.program_id(0)</code> identify?	The current program's coordinate on grid axis 0.	Program, not lane/thread.	Calling it a global thread index.
2. How do you map an ID to a vector block?	<code>offsets=pid*BLOCK+arange(0,BLOCK).</code>	Base plus local vector offsets.	Omitting <code>BLOCK</code> multiplication.

Question	Clear answer	Key points expected	Common mistake
3. How do 2D IDs map to a matrix?	One axis selects a row tile and the other a column tile.	Grid/tile bounds and strides.	Treating IDs as element row/column without tile scaling.
4. Why flatten a 2D grid?	It allows custom traversal, such as grouped ordering for L2 locality.	Decode with quotient/remainder.	Believing flattening reduces the number of programs.
5. What is <code>tl.num_programs</code> for?	It reports grid extent on an axis, useful as the step in grid-stride loops.	Persistent workload distribution.	Using block size as the step.
6. Do IDs define execution order?	No reliable global execution order should be assumed.	Programs are independent; scheduler decides.	Using order for synchronization.
7. Can two programs write the same address?	They can, but normal stores race; use disjoint ownership or appropriate atomics/reduction design.	Address mapping determines races.	Assuming different IDs make stores safe.
8. How do you handle more logical dimensions than grid axes?	Flatten dimensions and decode them, or combine dimensions in an axis.	Correct divisor/stride math.	Reversing batch/head decoding.
9. Why reorder matmul program IDs?	Nearby programs can reuse A/B tiles in L2, improving cache hit rate.	Locality, unchanged output coverage.	Claiming it changes arithmetic complexity.
10. How do you verify a mapping?	Test tiny dimensions, enumerate expected tiles/offsets, check complete coverage and no duplicate writes.	Boundary and ownership reasoning.	Testing only square/divisible inputs.

7. Deep-Dive Questions

1. **What makes a mapping load-balanced?** Each program should receive similar work. Variable-length rows may require bucketing, a work queue, or grid-stride scheduling instead of one heavy row per ID.
2. **How can grouped matmul ordering help?** It processes multiple neighboring M tiles while staying near an N group, increasing the chance that an operand tile remains in L2 for subsequent programs.
3. **Can program IDs synchronize?** No global barrier follows from IDs. Split dependent global phases into separate kernel launches or use an explicitly safe synchronization algorithm.

4. **What is a persistent grid?** It launches a hardware-sized/fixed number of programs; each program repeatedly takes new tiles, often using `num_programs` or a counter, reducing launch/scheduling overhead for suitable workloads.
5. **How does mapping affect coalescing?** IDs choose the tile; the `tl.arange` layout and pointer expression inside that tile decide whether neighboring lanes access adjacent addresses.

8. Comparison Tables

Mapping	Advantage	Cost/risk	Good use
1D consecutive	Simple, naturally coalesced	Limited multidimensional expression	Vectors
Native 2D grid	Clear tile coordinates	Less control over traversal order	Images/simple matrices
Flattened 2D	Custom ordering	More index math	Matmul locality
Grid-stride	Fixed program count, repeated work	Loop and possible imbalance	Persistent/large workloads
Triton term	Rough CUDA analogy	Important difference	
<code>tl.program_id</code>	<code>blockIdx</code>	Triton program handles blocked tensor values	
Grid tuple	CUDA grid dimensions	Counts Triton programs	
<code>tl.arange</code> offsets	Work distributed among block threads	Expressed as a block value, mapping compiler-managed	

9. Common Mistakes

- Confusing a program ID with a scalar element or hardware thread ID.
- Swapping grid axes between wrapper and kernel.
- Using floor division for grid size and losing the tail.
- Decoding a flat ID with the wrong number of tiles.
- Assuming program execution order or cross-program synchronization.
- Creating duplicate output ownership after a grouped-ordering optimization.

10. Edge Cases / Special Cases

- Boundary programs usually own partial tiles and require masks.
- A zero-length dimension should be handled before launch.
- Very large flattened grids require index arithmetic wide enough for products and offsets.
- Ragged data can produce severe imbalance under a simple one-ID-per-row mapping.
- Grouped formulas must handle a final group smaller than the normal group size.
- Atomics make overlapping ownership defined only when the operation and dtype are supported and the algorithm tolerates non-deterministic order.

11. How to Explain in Interview

“Every Triton program runs the same kernel, so `tl.program_id(axis)` gives it a coordinate in the launch grid. I multiply that coordinate by a tile size, add vector offsets, and use the result to own a disjoint tensor tile. For matrices I use two axes or decode a flat ID, sometimes reordering tiles for cache locality.”

12. Quick Revision Notes

- `program_id(axis)` = coordinate, not hardware thread.
- Vector base = `pid * BLOCK`.
- Flat 2D decode = quotient and remainder.
- Grid size commonly uses `triton.cdiv`.
- Never depend on program scheduling order.
- Verify coverage, uniqueness, boundary masks, and stride math.

13. Practice Tasks

1. List offsets for `n=19`, `BLOCK=8`, and all launched IDs.
2. Implement a row-wise scale kernel with a 2D grid.
3. Flatten an `(M_tiles, N_tiles)` grid and prove decode/encode are inverses.
4. Create a tiny script that records each program's intended output interval and detects overlap.
5. Derive a batch/head mapping for `[B,H,S,D]`.
6. Compare row-major and grouped matmul tile orders on paper.

14. Final Cheat Sheet

Item	Answer
Core definition	Coordinate of the current program in the launch grid
Why it matters	Assigns disjoint tensor tiles to identical program instances
Most asked	1D mapping, 2D decode, ordering, races, <code>num_programs</code>
Main comparison	Program ID resembles block ID, not element/thread ID
One-line answer	“Program IDs select the tile; local block offsets select elements inside it.”

Blocks

1. Overview

Definition. A Triton block is a compile-time-shaped vector or multidimensional tile of values processed by one program instance. It is typically built from `tl.arange`, broadcasting, pointer arithmetic, and loads.

Blocks matter because their shape determines memory access, available parallel work, reduction scope, register pressure, and reuse. They appear in every Triton kernel: vector chunks, rows for softmax, and `M×N` tiles for matrix multiplication. Interviewers ask how you choose a block size and why padding/masking are necessary.

2. Core Idea

A block is like carrying a tray of items rather than moving one item at a time. The program describes an operation on the whole tray; the compiler distributes that work over GPU lanes.

```
cols = tl.arange(0, BLOCK_N)          # [BLOCK_N]
rows = tl.arange(0, BLOCK_M)          # [BLOCK_M]
offsets = rows[:, None] * stride_m + cols[None, :] * stride_n
tile = tl.load(x_ptr + offsets, mask=valid, other=0.0) # [BLOCK_M, BLOCK_N]
```

For `BLOCK_M=2` and `BLOCK_N=4`, broadcasting forms eight addresses. The program can apply elementwise operations, reduce an axis, use `tl.dot`, and store a resulting tile. Block shapes are static so the compiler can plan layout and resources.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Block shape	Compile-time dimensions of a blocked value.	<code>[128], [32, 32]</code> .	Shape controls both parallelism and resource use.
<code>tl.arange</code>	Creates a compile-time interval along an axis.	<code>tl.arange(0, 256)</code> .	End/length constraints and static shape.
Broadcasting	Expands compatible dimensions without manually nested loops.	<code>rows[:, None] + cols[None, :]</code> .	Derive 2D pointer tiles.
Padding	Logical size is rounded to a legal/convenient block size.	Row length 1000, block 1024.	Padded lanes must not affect reductions.
Tiling	Divides a large tensor into reusable blocks.	<code>BM×BK</code> and <code>BK×BN</code> matmul tiles.	Reuse versus occupancy tradeoff.
Reductions	Combine a block axis inside a program.	<code>tl.sum(x, axis=0)</code> .	Masked identity values and numerical precision.
Register pressure	Larger/live blocks require more registers and may spill.	Fused operation keeps several tiles live.	“Larger is faster” is false.
Warp/stage tuning	Execution resources used for a block.	4 or 8 warps; pipelined K stages.	Values are benchmarked, not universal constants.

4. Real-World Example

A recommender system normalizes embedding rows of length 768. One Triton program can load a padded 1024-element block, mask the final 256 lanes, compute row statistics, normalize valid values, and store them. One-row-per-program gives a simple ownership model and keeps intermediate statistics local.

5. Diagrams / Mental Models

```

Logical row: [ x x x x x x ]           N = 6
Block:      [ x x x x x x P P ]       BLOCK = 8
Mask:      [ 1 1 1 1 1 1 0 0 ]

2D tile from broadcasting:
rows[:,None]      cols[None,:]      address grid
[0]                [0 1 2 3]         [00 01 02 03]
[1]                +                 [10 11 12 13]

```

6. Common Interview Questions

Question	Clear answer	Key points expected	Common mistake
1. What is a Triton block?	A statically shaped vector/tile operated on by one program.	Logical blocked value, compiler mapped.	Equating it exactly with a CUDA thread block.
2. Why are block sizes compile-time values?	Static shapes enable layout planning, unrolling, and efficient lowering.	<code>tl.constexpr</code> .	Passing an arbitrary runtime length to <code>tl.arange</code> .
3. Why use power-of-two blocks often?	Many Triton block/reduction patterns require or favor legal power-of-two shapes; padding simplifies arbitrary lengths.	Mask invalid lanes.	Claiming input dimensions must be powers of two.
4. How do you choose block size?	Start from access/reuse requirements, then benchmark legal candidates while watching registers, occupancy, and work size.	Tradeoffs, hardware dependence.	Always choosing 1024.
5. What happens if a block crosses a boundary?	Loads/stores are masked; invalid loaded values use a safe <code>other</code> suited to later math.	Correct mask and identity.	Masking store but not load.
6. How do you make a 2D block?	Broadcast row and column ranges into a matrix of offsets.	<code>[:,None]</code> and <code>[None,:]</code> .	Materializing coordinates on host.
7. Why can a larger tile help matmul?	It increases reuse of loaded A/B data and arithmetic intensity.	Fewer global bytes per result.	Ignoring higher resource consumption.
8. Why can it hurt?	More registers/on-chip storage can reduce occupancy or spill; edge waste also grows.	Resource/shape tradeoff.	Assuming compiler always splits it optimally.
9. Is a block stored in shared memory?	Not necessarily; it is an IR/value abstraction mapped by the compiler	Avoid false physical equivalence.	Saying every block is SRAM.

Question	Clear answer	Key points expected	Common mistake
	to registers and other storage as needed.		
10. How do reductions interact with padding?	Invalid lanes must be replaced by the reduction identity, such as 0 for sum and <code>-inf</code> for max.	Mathematical identity.	Using zero for max on all-negative data.

7. Deep-Dive Questions

1. **How does tile shape affect arithmetic intensity?** Larger output tiles reuse each loaded A value across more columns and each B value across more rows, but resource usage and boundary waste rise.
2. **What is live-range pressure?** Fusion may keep inputs, statistics, and outputs live simultaneously. Even if each block fits alone, overlapping lifetimes can cause register spills.
3. **Why are skinny and square tiles useful in different cases?** Shape should follow data layout, reduction direction, and dimensions. A row reduction favors a wide 1D tile; GEMM often benefits from balanced M/N reuse.
4. **Can one program process multiple blocks?** Yes, via loops/grid-stride work or persistent scheduling, but that changes register reuse, latency hiding, and load balancing.
5. **How should a reduction block exceed a feasible one-program width?** Use a hierarchical/multi-pass algorithm or tiled loop with partial accumulators; do not assume an arbitrarily wide row fits efficiently in one instance.

8. Comparison Tables

Aspect	Small block	Large block
Parallel programs	More	Fewer
Per-program reuse	Usually lower	Potentially higher
Registers/on-chip use	Lower	Higher
Boundary waste	Lower	Can be higher
Reduction reach	Smaller	Larger
Best choice	Benchmark under real shapes	Benchmark under real shapes

Term	Meaning
Logical tensor dimension	Actual valid data length
Block dimension	Static processing extent, often rounded up
Launch grid dimension	Number of independent program instances
Warp count	Execution resource parameter for each program

9. Common Mistakes

- Treating block size, grid size, and warp count as the same setting.
- Assuming a logical tensor must match the static block exactly.
- Using invalid padded values in a max, min, mean, or variance.
- Choosing tile size from folklore rather than measuring representative shapes.
- Ignoring registers and spills when fusing larger tiles.
- Calling every blocked value “shared memory.”

10. Edge Cases / Special Cases

- A row longer than a practical single block may need chunked or two-pass reduction.
- Odd dimensions can waste much of a large padded block.
- A mask's shape must broadcast exactly to the pointer block.
- For a mean, invalid lanes contribute zero to the sum but the divisor remains the logical count, not padded block size.
- Integer address multiplication can overflow if unnecessarily narrow types are used for huge tensors.
- **other** may be converted to the loaded element type; choose values representable in the relevant computation path.

11. How to Explain in Interview

“A Triton block is a compile-time-shaped vector or tile handled by one program. I form it with ranges and broadcasting, use masks when the logical tensor is smaller, and choose its shape to balance coalescing and reuse against register pressure and occupancy.”

12. Quick Revision Notes

- Blocks are logical values, not guaranteed physical shared-memory arrays.
- Shapes are compile-time known.
- Padding handles arbitrary logical sizes.
- Reduction padding must use the correct identity.
- Larger tile: more reuse, but more registers and boundary waste.
- Benchmark tile/warp/stage combinations on representative inputs.

13. Practice Tasks

1. Draw the pointer matrix produced by 3 rows and 4 columns with arbitrary strides.
2. Implement row sum for **N=1000** using a 1024-wide block and prove padding is harmless.
3. Change row max input to all-negative values and test the invalid-lane identity.
4. Benchmark vector blocks 128, 256, 512, and 1024.
5. Estimate bytes and FLOPs for several matmul tile shapes.
6. Inspect compiler/profiler output for spills after increasing a fused tile.

14. Final Cheat Sheet

Item	Answer
Core definition	Static vector/tile processed by one Triton program
Why it matters	Defines access pattern, reuse, reduction scope, and resource demand

Item	Answer
Most asked	Static shapes, padding, tile choice, broadcasting, reductions
Main comparison	Large blocks improve reuse but can reduce occupancy/spill
One-line answer	"Choose the smallest block that expresses useful parallel work and reuse, then benchmark."

Masked Loads and Stores

1. Overview

Definition. A mask is a Boolean block passed to `tl.load` or `tl.store`. A masked load reads only positions where the mask is true and supplies `other` for false positions; a masked store writes only true positions.

Masks matter because fixed compile-time blocks rarely divide arbitrary tensor dimensions exactly. They prevent out-of-bounds memory access, express triangular/causal structure, and support ragged or selectively valid data. Interviewers ask masks because a kernel that works only for divisible shapes is incomplete, and a wrong masked identity silently corrupts reductions.

2. Core Idea

A mask is a stencil placed over addresses: open holes access memory; closed holes do not.

```
offsets = pid * BLOCK + tl.arange(0, BLOCK)
mask = offsets < n
x = tl.load(x_ptr + offsets, mask=mask, other=0.0)
tl.store(y_ptr + offsets, x * 2, mask=mask)
```

For `n=10`, `BLOCK=8`, program 1 generates offsets `[8,9,10,11,12,13,14,15]` and mask `[T,T,F,F,F,F,F,F]`. Only `x[8]` and `x[9]` are read or written. `other=0` is a safe placeholder for this elementwise computation.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Boundary mask	Guards tail elements outside a logical dimension.	<code>offs < n.</code>	Required even if allocation happens to be larger.
Load <code>other</code>	Value returned for masked-off load lanes.	<code>0</code> for sums, <code>-inf</code> for max.	Choose according to subsequent operation.
Store mask	Suppresses writes for invalid lanes.	<code>tl.store(ptr, value, mask).</code>	A masked store does not make an earlier invalid load safe.

Subtopic	Meaning and why it matters	Example	Common interview angle
Multidimensional mask	Combines validity per axis with Boolean operators.	<code>(rows<M)[: ,None] & (cols<N)[None, :]</code> .	Parentheses/broadcasting and both boundaries.
Semantic mask	Represents algorithmic validity, not only tensor tails.	Causal attention: key index $\leq$ query index.	Masking implements mathematical structure.
NaN/Inf interaction	Masked placeholders and <code>tl.where</code> placement affect reductions.	<code>-inf</code> before row max.	<code>0 * NaN</code> is still NaN; do not rely on multiplication to mask.
Block pointers	Some pointer APIs can express boundary checking/padding by dimensions.	Boundary-aware block load.	Know that API convenience does not remove identity reasoning.

4. Real-World Example

In autoregressive attention, query token `i` must not see future key tokens `j>i`. A kernel forms a score tile and sets future positions to `-inf` before softmax. The same tile also needs bounds masks when the final sequence block is partial. One mask enforces model semantics; another enforces memory safety, and both must be correct.

5. Diagrams / Mental Models

Tail mask, n=6, BLOCK=8
offset: 0 1 2 3 4 5 6 7
valid: T T T T T T F F
load: x x x x x x 0 0

Causal mask for four tokens (row=query, col=key)
 k0 k1 k2 k3
q0 T F F F
q1 T T F F
q2 T T T F
q3 T T T T

Later operation	Safe invalid load value
Sum/add	<code>0</code>
Product	<code>1</code>
Maximum	<code>-inf</code> or lowest representable value
Minimum	<code>+inf</code> or highest representable value

Later operation Safe invalid load value

Softmax score $-\text{inf}$ before exponentiation

6. Common Interview Questions

Question	Clear answer	Key points expected	Common mistake
1. Why are masks needed?	Static tiles can extend beyond logical bounds; masks prevent invalid memory operations and represent selective validity.	Safety plus semantics.	Saying padding alone makes reads legal.
2. What does other do?	It supplies the result for false lanes of a masked load.	It influences later math.	Thinking it initializes out-of-bounds memory.
3. Is a store mask enough?	No. Any preceding load must independently avoid invalid addresses.	Load and store safety are separate.	Loading OOB then masking only the output.
4. What value should a masked max load use?	$-\text{inf}$, the identity for maximum.	All-negative inputs remain correct.	Zero, which can become a false maximum.
5. How do you mask a matrix tile?	Broadcast per-row and per-column conditions and combine them with & .	Shape alignment.	Using Python and instead of elementwise & .
6. Does a false masked load access memory?	Semantically no; that lane yields other .	This is what makes boundary access safe.	Assuming the pointer itself may be arbitrary in every API/context without considering address formation constraints.
7. How is a causal mask different from a boundary mask?	Causal masking enforces allowed dependencies; boundary masking protects physical/logical tensor limits.	Both may be combined.	Treating future positions as merely padding.
8. How do masks affect softmax?	Invalid logits must behave as $-\text{inf}$, so their exponentials are zero and they do not change max or sum.	Stable softmax identities.	Masking after normalization only.
9. Can masks prevent races?	They can assign disjoint subsets, but race freedom depends on global ownership/address mapping, not the presence of a mask alone.	Cross-program reasoning.	Assuming any masked store is race-free.

Question	Clear answer	Key points expected	Common mistake
10. What should tests cover?	Sizes below, equal to, and just above block boundaries; multidimensional tails; all-masked or semantic-mask cases where valid.	Adversarial shapes.	Only multiples of block size.

7. Deep-Dive Questions

1. **Why can post-load `tl.where(mask, x, 0)` be unsafe?** If `x` came from an unmasked out-of-bounds load, the invalid memory operation already occurred. Mask the load itself.
2. **Why can masking after `exp` be numerically wrong?** An invalid large score may affect the row max or overflow before it is zeroed. Replace it with `-inf` before max/exponentiation.
3. **What happens for an all-masked softmax row?** Max may be `-inf`, leading to `-inf - -inf = NaN`, and the denominator is zero. The wrapper/kernel needs an explicit policy such as returning zeros or rejecting the shape.
4. **Can predication cost performance?** Boundary masks are usually cheap relative to unsafe special cases, but heavily irregular masks can reduce useful work and memory efficiency. Bucketing or specialized interior kernels may help only when measurement justifies them.
5. **What is the distinction between mask and padding?** Padding changes or reserves storage/layout; a mask changes which lanes participate. A logical padded tile still needs correct access bounds unless physical padding is guaranteed and intended.

8. Comparison Tables

Technique	Purpose	Advantage	Risk
Masked load/store	Per-lane safe/semantic access	One kernel handles arbitrary sizes	Invalid identity must be correct
Physically pad tensor	Make dimensions regular in memory	Simpler repeated kernels	Extra memory/copy; padding may leak into math
Separate tail kernel	Handle remainder independently	Main path may avoid masks	Extra launch/code and maintenance

Require divisibility	Narrow contract	Simplest kernel	Often unacceptable for general callers
Boundary mask	Semantic mask		
Prevents out-of-range access	Prevents disallowed logical participation		
Based on shape bounds	Based on algorithm, e.g. causality		
Usually only partial tiles differ	May create structure across every tile		

9. Common Mistakes

- Masking only stores while leaving out-of-bounds loads.
- Using zero as the identity for max/min reductions.
- Dividing means by padded block size instead of valid count.
- Using scalar Python **and/or** instead of elementwise Boolean operations.
- Applying a softmax mask after max or exponentiation.
- Forgetting one dimension in a 2D boundary mask.
- Assuming a mask fixes overlapping writes between programs.

10. Edge Cases / Special Cases

- **n=0** is best rejected/returned early by the host wrapper.
- All-masked reductions need a defined result or explicit invalid-input rule.
- NaNs in valid data propagate according to the chosen operation; masking is not general NaN sanitization.
- Infinity placeholders must be representable or safely converted in the computation dtype.
- Broadcast masks can appear shape-compatible while guarding the wrong dimension; test rectangular tiles.
- For causal attention with unequal query/key positions, include sequence offsets, not only local tile coordinates.

11. How to Explain in Interview

"Triton uses fixed block shapes, so a boundary block often contains invalid lanes. I mask every potentially invalid load and store, and choose the load's **other** as the identity of the next operation—zero for sum, negative infinity for max or masked softmax. Masks can also encode semantics such as causal attention."

12. Quick Revision Notes

- Mask loads **and** stores as required.
- **other** is a computed lane value, not a memory write.
- Sum identity **0**; max identity **-inf**; min identity **+inf**.
- Combine 2D conditions with broadcasted **&**.
- Mask before reductions/nonlinear operations.
- All-masked rows require an explicit policy.

13. Practice Tasks

1. Run vector addition for lengths **0,1,BLOCK-1,BLOCK,BLOCK+1**.
2. Implement row max with all-negative values and a partial final block.
3. Draw and code a combined rectangular-boundary plus causal mask.
4. Deliberately use zero for masked max, find the failing test, then correct it.
5. Test a mean kernel where **N** is not a power of two.
6. Explain why multiplying invalid values by zero is not a safe substitute for masked loading.

14. Final Cheat Sheet

Item	Answer
Core definition	Boolean per-lane guard for memory operations
Why it matters	Safe arbitrary shapes and semantic selection
Most asked	Load vs store masks, other , reduction identity, causal masking
Main comparison	Masking avoids physical padding but still performs padded-lane compute
One-line answer	"Mask invalid memory lanes and substitute the mathematical identity before any reduction."

Vector Addition

1. Overview

Definition. Vector addition computes $z[i] = x[i] + y[i]$ independently for every element. In Triton, each program handles a consecutive block of indices rather than a single scalar.

It is the "hello world" of Triton because it introduces JIT kernels, launch grids, program IDs, vector offsets, and masked memory operations without requiring synchronization or reductions. The same pattern appears in residual connections, tensor updates, simulation arrays, and preprocessing. Interviewers ask it to expose indexing mistakes and to see whether you recognize a memory-bandwidth-bound kernel.

2. Core Idea

Imagine splitting two long lists into equal-size packets. Each worker receives packet **pid**, adds corresponding entries, and returns one packet. No worker needs another worker's result.

```
import torch
import triton
import triton.language as tl

@triton.jit
def add_kernel(x_ptr, y_ptr, out_ptr, n,
               BLOCK: tl.constexpr):
    offsets = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    tl.store(out_ptr + offsets, x + y, mask=mask)

def triton_add(x: torch.Tensor, y: torch.Tensor):
    assert x.is_cuda and y.is_cuda and x.shape == y.shape
    out = torch.empty_like(x)
    n = out.numel()
    if n:
```

```
add_kernel[(triton.cdiv(n, 256),)](x, y, out, n, BLOCK=256)
return out
```

For $n=10$, $BLOCK=8$, two programs are launched. Program 0 owns elements 0–7; program 1 generates 8–15 but its mask permits only 8 and 9. Because adjacent lanes use adjacent addresses, loads and stores are coalescing-friendly.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Element independence	Each output depends only on same-index inputs.	$z[7]=x[7]+y[7]$.	No inter-program communication needed.
Ceiling grid	Launch enough blocks for every element.	$\text{cdiv}(n, BLOCK)$.	Tail requires masking.
Coalescing	Consecutive lanes access consecutive elements.	$\text{ptr} + \text{offsets}$.	Good access pattern, not just contiguous allocation.
Arithmetic intensity	One add for two reads and one write is little computation per byte.	FP32: about 1 FLOP/12 bytes.	Usually memory bandwidth bound.
In-place aliasing	Output may alias an input only if per-index reads occur before writes and no cross-index dependence exists.	$\text{out}=\text{x}$ can be valid for pure add.	Wrapper/framework semantics and compiler assumptions still matter.
Launch overhead	Fixed dispatch cost dominates small vectors.	128 elements may favor existing/fused CPU or GPU operation.	End-to-end latency versus peak bandwidth.
Benchmarking	Measure steady state and effective bandwidth.	$\text{bytes}=3*n*\text{element_size}$.	Warm-up and asynchronous execution.

4. Real-World Example

Transformer residual connections compute $\text{hidden} = \text{hidden} + \text{sublayer_output}$. A standalone vector add reads two tensors and writes a third. If the next operation is dropout or normalization, a production kernel may fuse the addition with it to avoid materializing and rereading the intermediate. Vector addition therefore teaches both the basic mapping and why fusion matters.

5. Diagrams / Mental Models

```

Grid, BLOCK=4
Program 0: offsets [0 1 2 3]
Program 1: offsets [4 5 6 7]

x:  [x0 x1 x2 x3 | x4 x5 x6 x7]
    + + + +   + + + +
y:  [y0 y1 y2 y3 | y4 y5 y6 y7]
    = = = =   = = = =
out: [z0 z1 z2 z3 | z4 z5 z6 z7]

```

FP32 work per element	Amount
Loads	8 bytes
Stores	4 bytes
Arithmetic	1 addition
Approximate intensity	1/12 FLOP per byte

6. Common Interview Questions

Question	Clear answer	Key points expected	Common mistake
1. How many programs are launched?	<code>ceil(n/BLOCK)</code> for a one-block-per-chunk mapping.	Ceiling division.	<code>n//BLOCK</code> , losing the tail.
2. How does each program find its elements?	Base is <code>program_id(0)*BLOCK</code> , then add <code>tl.arange</code> .	Blocked indexing.	Treating ID as element index.
3. Why is a mask needed?	The last block may extend beyond <code>n</code> .	Safe arbitrary length.	Assuming input is divisible.
4. Why is vector add bandwidth-bound?	It transfers 12 FP32 bytes for only one addition per element.	Arithmetic intensity/roofline.	Focusing on ALU throughput.
5. What is effective bandwidth?	Useful bytes moved divided by elapsed time; commonly count two input reads and one output write.	<code>3*n*element_size/time</code> .	Reporting GB/s without defining counted bytes.
6. Why might PyTorch be just as fast?	Its elementwise kernel is already simple and optimized; both may saturate memory bandwidth.	Triton is educational here.	Claiming custom always wins.

Question	Clear answer	Key points expected	Common mistake
7. Can vector add be in-place?	Mathematically yes for same-index aliasing, but the API must permit it and overlapping non-identical views can race.	Alias contract.	Unconditional yes for all layouts.
8. Does shared memory help?	Usually no because each input is used once; staging adds traffic and synchronization without reuse.	Reuse criterion.	"Shared memory is faster" without cost analysis.
9. How should it be timed?	Warm up/JIT first, use GPU-aware timing or synchronize around measurement, repeat, and test multiple sizes.	Asynchronous launches.	Timing only the Python call with a CPU clock.
10. How can it be improved?	Preserve coalescing, choose a sensible block, keep data resident, and fuse adjacent work; there is little arithmetic to optimize.	Traffic/launch reduction.	Adding complicated math or huge tiles.

7. Deep-Dive Questions

1. **What does the roofline model predict?** With roughly **1/12** FP32 FLOP per byte, attainable FLOP/s is approximately memory bandwidth times that low intensity, far below peak compute.
2. **Would wider/vectorized loads reduce time?** They may reduce instruction overhead when alignment and shape permit, but not bytes transferred. Once bandwidth is saturated, improvement is limited.
3. **What changes for broadcasting?** Address expressions use zero or different strides for broadcasted dimensions; naive flattening may read the wrong element and may lose coalescing.
4. **Why can a fused add be much faster than standalone add?** Fusion avoids an intermediate global write/read and possibly a launch, which attacks the actual bottleneck.
5. **How do overlapping views break in-place safety?** If an output address is also an input address for another index/program, a write can occur before that input is read, producing a race.

8. Comparison Tables

CPU loop	Triton vector-add kernel
Very low invocation overhead	GPU launch/JIT warm-up cost
Limited memory bandwidth/parallelism	High device bandwidth and many lanes
Best when data is on CPU or tiny	Best when large tensors already reside on GPU
Ordinary scalar/vector compiler code	Explicit blocked GPU mapping
Standalone add	Fused add + next operation
Reusable and simple	Specialized
Writes intermediate output	May keep intermediate on chip

Standalone add	Fused add + next operation
One launch	Avoids an additional launch
Often bandwidth-bound	Higher useful work per byte

9. Common Mistakes

- Using the program ID directly as a scalar index.
- Launching `n` programs and processing `BLOCK` values in each.
- Forgetting the final mask or using floor division.
- Claiming compute optimization matters more than memory traffic.
- Adding shared-memory staging despite zero reuse.
- Benchmarking the first call, which includes compilation.
- Ignoring non-contiguous layouts or unsafe aliasing.

10. Edge Cases / Special Cases

- `n=0`: return without launching.
- `n<BLOCK`: one mostly masked program is correct but inefficient for extremely small work.
- Mixed dtypes: define promotion/output semantics explicitly; do not assume PyTorch's rules automatically.
- Very large tensors: use sufficiently wide offset calculations and avoid shape-product overflow in the host.
- Non-contiguous inputs: either accept strides and calculate addresses or validate a contiguous-only contract.
- Device mismatch: pointers from different devices cannot be used by one kernel.

11. How to Explain in Interview

"I launch `ceil(n/BLOCK)` Triton programs. Each program uses its ID to create a consecutive vector of offsets, masks the tail, loads two vectors, adds them, and stores the result. The kernel is normally memory-bound—one add costs two reads and one write—so coalescing and fusion matter more than arithmetic tuning."

12. Quick Revision Notes

- Formula: `out[i]=x[i]+y[i]`.
- Grid: `(triton.cdiv(n,BLOCK),)`.
- Offsets: `pid*BLOCK + tl.arange(0,BLOCK)`.
- Tail: `offsets<n` on loads and store.
- FP32 traffic: approximately 12 bytes/element.
- Main optimization: coalescing, residency, fusion.
- Trap: first-call and unsynchronized timing.

13. Practice Tasks

1. Implement and validate the kernel for lengths around every block boundary.
2. Add scalar broadcasting: `out=x+alpha`.
3. Accept independent input/output strides and test sliced tensors.

4. Measure effective GB/s from 1 KiB to hundreds of MiB.
5. Fuse addition with ReLU and calculate bytes saved.
6. Test safe exact aliasing and construct an unsafe overlapping-view example conceptually.

14. Final Cheat Sheet

Item	Answer
Core definition	Independent same-index elementwise addition
Why it matters	Minimal complete example of Triton mapping and memory behavior
Most asked	Grid, offsets, tail mask, coalescing, bandwidth bound
Main comparison	Standalone add moves an intermediate; fusion can remove that traffic
One-line answer	"Vector add is a masked, coalesced blocked map whose speed is normally limited by memory bandwidth."

Softmax

1. Overview

Definition. Softmax converts logits in a row into non-negative values summing to one:

$$\text{softmax}(x_i) = \exp(x_i - \max(x)) / \sum_j \exp(x_j - \max(x)).$$

Subtracting the maximum is mathematically equivalent but prevents overflow. Triton's fused row-wise softmax is important because a naive implementation launches separate max, subtraction, exponentiation, sum, and division kernels, repeatedly moving data through global memory. It is used in classification, attention, routing, and probabilistic models. Interviewers ask it to test reductions, numerical stability, masks, resource limits, and fusion.

2. Core Idea

Think of scores as heights. First lower every height by the tallest one; none is now positive, so exponentials cannot overflow. Convert heights into positive weights, add the weights, and divide each by the total.

```
@triton.jit
def softmax_kernel(x_ptr, y_ptr, x_stride, y_stride, n_cols,
                  BLOCK: tl.constexpr):
    row = tl.program_id(0)
    cols = tl.arange(0, BLOCK)
    mask = cols < n_cols
    x = tl.load(x_ptr + row * x_stride + cols,
               mask=mask, other=-float("inf")).to(tl.float32)
    x = x - tl.max(x, axis=0)
    numer = tl.exp(x)
```

```
denom = tl.sum(number, axis=0)
tl.store(y_ptr + row * y_stride + cols, numer / denom, mask=mask)
```

One program owns one row. The host selects `BLOCK=triton.next_power_of_2(n_cols)` and launches one program per row. Invalid padded lanes load `-inf`; after exponentiation they become zero and do not change the denominator.

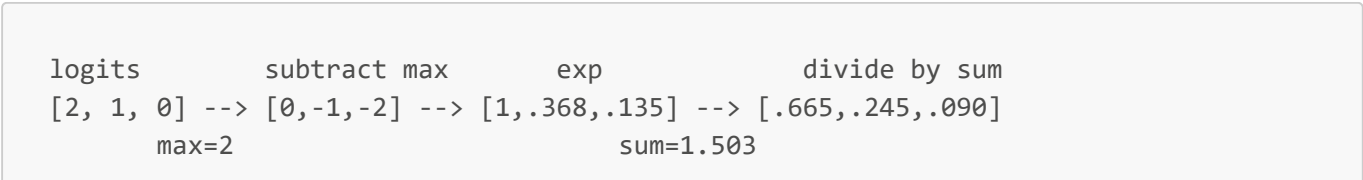
3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Stable softmax	Subtract row max before exponentiating.	<code>[1000,1001]</code> becomes <code>[-1,0]</code> .	Prevent overflow without changing result.
Row reduction	Max and sum combine every column in a row.	<code>tl.max, tl.sum</code> .	One program can reduce a block locally.
Padding identity	Invalid logits act as <code>-inf</code> ; exponentials become zero.	Row 1000 padded to 1024.	Zero is wrong before max for negative rows.
FP32 compute	Upcast lower-precision logits for max/exp/sum.	FP16 input, FP32 intermediates.	Stability versus output dtype/performance.
Fusion	Keep row/intermediates on chip and write final output once.	Load once, store once.	Traffic savings drive speedup.
Row stride	Address each row even when leading dimension has padding.	<code>row*x_stride+cols</code> .	Contiguity along normalized axis is important.
Large rows	A whole row may exceed efficient per-program resources.	Very large vocabulary/sequence.	Need tiled or multi-pass/online softmax.
Online softmax	Maintains running max and normalized sum across tiles.	Merge <code>(m,1)</code> statistics.	Supports long rows/attention without materialization.

4. Real-World Example

In transformer attention, each query produces scores against many keys. Softmax normalizes those scores before multiplying by values. A fused attention kernel goes further than standalone softmax: it tiles score computation, maintains online max/sum statistics, and accumulates the value-weighted output without storing the full score matrix. This reduces memory from materializing an `SxS` intermediate.

5. Diagrams / Mental Models



Global memory: X row --load once--> [program-local work] --store once--> Y row
 max -> exp -> sum -> divide

Implementation	Approximate global passes/intermediates
Naive separate operators	Multiple reads/writes for max, exp, sum, divide
Fused row softmax	One input read + one output write for fitting rows

6. Common Interview Questions

Question	Clear answer	Key points expected	Common mistake
1. Why subtract the maximum?	It prevents exponential overflow; softmax is unchanged because the common factor cancels.	Numerical proof/intuition.	Saying it prevents all underflow or changes probabilities.
2. Why load invalid lanes as <code>-inf</code> ?	They cannot become the max, and <code>exp(-inf)=0</code> , so they do not affect the sum.	Correct reduction identity.	Loading zero, wrong for all-negative rows.
3. Why is fusion helpful?	It avoids storing and rereading intermediate max-adjusted/exponential values and reduces launches.	Memory-bound traffic.	Claiming fewer floating-point operations.
4. How many programs?	Commonly one per row when a row fits efficiently in one program; persistent variants may process multiple rows.	Mapping depends on row feasibility.	One program per element.
5. Why compute in FP32?	Max, exponentiation, and summation are more stable; output may be cast back.	Mixed precision.	Assuming FP16 reduction is always adequate.
6. What does output sum to?	Ideally one; finite precision gives a value close to one.	Tolerance and rounding.	Requiring exact bitwise 1.0.
7. What limits one-row-per-program?	Static padded width, registers/on-chip storage, occupancy, and compiler/hardware limits.	Large-row fallback.	Assuming arbitrary rows fit.
8. What is online softmax?	A tiled algorithm that updates a running max and rescaled exponential sum, enabling exact stable softmax without storing all logits at once.	Correct rescaling when max changes.	Summing tile softmax outputs directly.
9. How does causal masking work?	Set disallowed logits to <code>-inf</code> before max and exp.	Semantic mask timing.	Zeroing probabilities only after softmax

Question	Clear answer	Key points expected	Common mistake
			without renormalizing.
10. How do you validate softmax?	Compare with a trusted implementation across shapes/dtypes/extreme logits/masks; check finite outputs and row sums within tolerance.	Adversarial numerical tests.	Random small logits only.

7. Deep-Dive Questions

1. **Prove shift invariance.** $\exp(x_i - c) / \sum \exp(x_j - c) = \exp(x_i) \exp(-c) / (\exp(-c) \sum \exp(x_j))$, so the common factor cancels.
2. **How do online statistics merge?** For old (m, l) and tile (m_t, l_t) , set $m_{\text{new}} = \max(m, m_t)$ and $l_{\text{new}} = l * \exp(m - m_{\text{new}}) + l_t * \exp(m_t - m_{\text{new}})$. Rescale any accumulated weighted output similarly.
3. **Why can all-masked rows produce NaN?** The max is `-inf`; subtracting it from `-inf` is undefined, and the exponential sum is zero. Define a zero output or forbid such rows.
4. **What is the softmax backward formula?** For upstream dy and output y , $dx = y * (dy - \sum(dy * y))$; it is another row reduction and is often fused.
5. **When might recomputation beat saving probabilities?** In memory-constrained fused attention backward, recomputing cheap scores/normalization statistics can save large intermediate storage and memory bandwidth.

8. Comparison Tables

Naive softmax	Fused Triton softmax
Several operator launches	One kernel launch for fitting rows
Materialized intermediates	Intermediates kept program-local
More global-memory traffic	One main read and write
Framework/compiler may fuse automatically	Explicit control and shape contract
Handles general shapes via library implementation	Custom kernel needs fallback/tuning
Standard row softmax	Online/tiled softmax
Holds a full padded row	Processes chunks and maintains statistics
Simpler	More state/rescaling logic
Excellent for moderate widths	Better for very long rows/fused attention
One local max and sum	Running max and sum merged across tiles

9. Common Mistakes

- Computing $\exp(x)$ before subtracting the maximum.
- Using zero for padded logits before the max reduction.

- Masking causal positions after normalization.
- Dividing by padded length rather than exponential sum.
- Keeping a huge row in one program and causing spills/poor occupancy.
- Testing only moderate random logits.
- Assuming a row sum must be exactly one in floating point.

10. Edge Cases / Special Cases

- All-masked row: define behavior explicitly.
- `n_cols=0`: invalid for ordinary softmax; reject or return an agreed empty result on host.
- `+inf` logits: subtracting `+inf` can create `NaN`; framework-compatible behavior may need explicit handling.
- `NaN` input: usually propagates; match the reference contract.
- Extreme negative differences may underflow to zero, which is generally acceptable and preferable to overflow.
- Non-contiguous normalized dimension can destroy coalescing; transpose/materialize or use strides based on measured tradeoffs.
- Very long rows need multi-stage or online algorithms.

11. How to Explain in Interview

“A fused Triton softmax typically maps one program to one row, loads a padded block with invalid lanes set to negative infinity, subtracts the row maximum, exponentiates, sums, divides, and stores once. It is numerically stable and avoids the multiple global-memory passes of separate framework operators. For rows too large to fit efficiently, I use a tiled online or multi-pass algorithm.”

12. Quick Revision Notes

- Formula: $\exp(x - \max(x)) / \sum(\exp(x - \max(x)))$.
- `-inf` is the invalid-logit identity.
- FP32 intermediates improve lower-precision stability.
- Fusion saves memory passes, not mathematical work.
- One program per row only while a row fits efficiently.
- Backward: $dx = y * (dy - \sum(dy * y))$.
- Trap: all-masked row and extreme infinities.

13. Practice Tasks

1. Implement fused row softmax and compare it to `torch.softmax`.
2. Test widths `1, 31, 32, 33, 1000, 1024, 1025` and extreme logits.
3. Add a causal or arbitrary Boolean mask before the max.
4. Implement softmax backward from the compact formula.
5. Derive and code online (`max, sum`) merging over two tiles.
6. Benchmark fused versus an intentionally multi-pass implementation and calculate bytes avoided.

14. Final Cheat Sheet

Item	Answer
Core definition	Stable exponential normalization over a row
Why it matters	Reduction-heavy primitive where fusion avoids repeated memory traffic
Most asked	Max subtraction, <code>-inf</code> mask, FP32 accumulation, large rows, backward
Main comparison	Full-row softmax is simple; online softmax scales through tiles
One-line answer	"Load once, mask with <code>-inf</code> , subtract max, exp, reduce sum, normalize, and store once."

Matrix Multiplication

1. Overview

Definition. Matrix multiplication (GEMM) computes $C = A @ B$, where A has shape $M \times K$, B has shape $K \times N$, and each output is $C[m,n] = \sum_k A[m,k]B[k,n]$.

GEMM dominates neural-network linear layers, convolutions lowered to matrices, attention projections, scientific computing, and graphics. Triton matmul matters because it demonstrates multidimensional pointer arithmetic, tiling, reuse, tensor/matrix-core operations, accumulation precision, cache-aware scheduling, and autotuning. Interviewers rarely expect a vendor-library replacement from memory; they expect the blocked algorithm and its tradeoffs.

2. Core Idea

Computing each output scalar independently would reread the same rows and columns repeatedly. Instead, tile the output. One Triton program owns a $BLOCK_M \times BLOCK_N$ tile of C , walks through K in $BLOCK_K$ chunks, loads an A tile and B tile, and accumulates their dot products.

```
@triton.jit
def matmul_kernel(a, b, c, M, N, K,
                  stride_am, stride_ak, stride_bk, stride_bn,
                  stride_cm, stride_cn,
                  BM: tl.constexpr, BN: tl.constexpr, BK: tl.constexpr):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    offs_m = pid_m * BM + tl.arange(0, BM)
    offs_n = pid_n * BN + tl.arange(0, BN)
    offs_k = tl.arange(0, BK)

    a_ptrs = a + offs_m[:, None] * stride_am + offs_k[None, :] * stride_ak
    b_ptrs = b + offs_k[:, None] * stride_bk + offs_n[None, :] * stride_bn
    acc = tl.zeros((BM, BN), tl.float32)

    for k0 in range(0, tl.cdiv(K, BK)):
        k = k0 * BK + offs_k
```

```

    a_tile = tl.load(a_ptrs, mask=(offs_m[:, None] < M) & (k[None, :] < K),
other=0.0)
    b_tile = tl.load(b_ptrs, mask=(k[:, None] < K) & (offs_n[None, :] < N),
other=0.0)
    acc += tl.dot(a_tile, b_tile)
    a_ptrs += BK * stride_ak
    b_ptrs += BK * stride_bk

c_ptrs = c + offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn
tl.store(c_ptrs, acc, mask=(offs_m[:, None] < M) & (offs_n[None, :] < N))

```

The launch grid is $(\text{ceil}(M/BM), \text{ceil}(N/BN))$. This simplified kernel emphasizes correctness. Production kernels specialize layouts, scheduling, dtypes, activation/epilogue behavior, and hardware tuning.

3. Important Subtopics

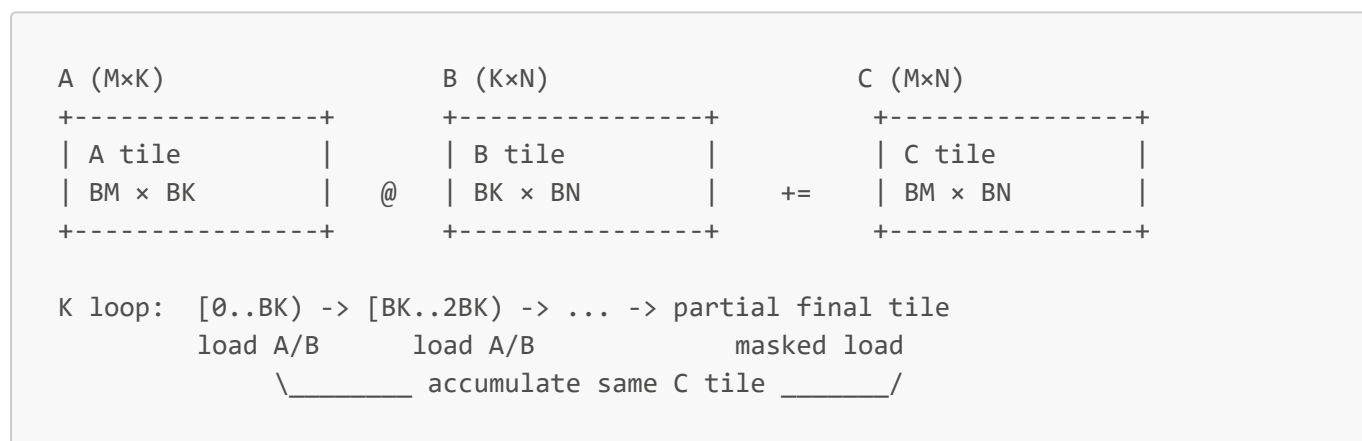
Subtopic	Meaning and why it matters	Example	Common interview angle
Output tiling	One program accumulates a rectangular C tile.	$BM \times BN$.	Independent ownership avoids output races.
K blocking	Reduction dimension is processed in chunks.	Loop over BK .	Accumulator persists; A/B pointers advance.
Data reuse	Each A tile value contributes to many N outputs; each B value to many M outputs.	A $BM \times BK$, B $BK \times BN$.	Raises arithmetic intensity.
Strides	Convert logical coordinates to physical addresses.	$m * \text{stride_am} + k * \text{stride_ak}$.	Support transposed/leading dimensions correctly.
Boundary masks	Zero-pad partial M/N/K tiles mathematically.	Invalid K loads use 0.	K mask differs from C-store mask.
<code>tl.dot</code>	Expresses block dot product, allowing accelerator lowering when eligible.	FP16/BF16 inputs, FP32 accumulator.	Dtype/tile constraints are hardware-dependent.
Accumulation dtype	Products may be low precision but summed in FP32.	Tensor-core mixed precision.	Accuracy/performance tradeoff.
Tile ordering	Program IDs can be grouped to improve L2 reuse.	Process nearby M tiles per N group.	Scheduling/locality without changing results.

Subtopic	Meaning and why it matters	Example	Common interview angle
Autotuning	Benchmarks candidate tile/warp/stage configurations per key shapes.	<code>@triton.autotune.</code>	Candidate set and key design matter; tuning has cost.
Epilogue fusion	Apply bias, activation, scaling, or quantization before C store.	<code>C=gelu(A@B+bias).</code>	Saves C intermediate traffic/launches.

4. Real-World Example

A transformer feed-forward layer computes $Y = \text{GELU}(XW + b)$. XW is GEMM; the bias and activation are an epilogue. A Triton matmul kernel can accumulate the C tile in FP32, add the matching bias columns, apply GELU, cast to the requested output type, and store once. This avoids a separate bias/activation read-write pass while preserving the high-reuse matmul core.

5. Diagrams / Mental Models



Tile work	Approximate count
FLOPs per K tile	$2 * BM * BN * BK$
A elements loaded	$BM * BK$
B elements loaded	$BK * BN$
C accumulator elements	$BM * BN$

6. Common Interview Questions

Question	Clear answer	Key points expected	Common mistake
1. Why tile matrix multiplication?	Tiling reuses A/B values for many multiply-adds, raising arithmetic	Reuse, not merely parallelism.	One program per output scalar.

Question	Clear answer	Key points expected	Common mistake
	intensity and enabling matrix hardware.		
2. What does one program compute?	Usually one $BM \times BN$ output tile accumulated across K.	Disjoint C ownership and K loop.	One K tile is the final result.
3. Why is there a loop over K?	Every C value is a reduction over all K; each iteration adds one partial block product.	Persistent FP32 accumulator.	Overwriting instead of accumulating.
4. How are A/B addresses formed?	Broadcast row/K and K/column offsets and multiply each logical axis by its stride.	2D pointer arithmetic.	Assuming every matrix is tightly row-major.
5. What masks are needed?	A guards M and K, B guards K and N, C store guards M and N. Invalid input elements contribute zero.	Correct axes and identities.	Reusing one incorrectly shaped mask.
6. Why accumulate in FP32?	It reduces error when multiplying FP16/BF16 values and matches common accelerator behavior.	Mixed-precision accumulation.	Claiming it makes the entire operation exact.
7. How do tile sizes affect performance?	Larger tiles improve reuse but increase registers/on-chip use and may reduce occupancy or waste edge work.	Tradeoff and benchmarking.	Universal “best” tile.
8. What is grouped program ordering?	A mapping that visits tiles in groups to improve L2 reuse of A or B across programs.	Locality only; same coverage.	Confusing it with K blocking.
9. Why use autotuning?	The best tile, warp, and stage configuration varies with shape, dtype, layout, and GPU.	Curated candidates and tuning keys.	Huge unbounded search or tuning every call.
10. When should you not write custom matmul?	For plain GEMM, use vendor/framework libraries unless profiling shows a custom shape/epilogue wins.	cuBLAS/rocBLAS are highly tuned.	Reimplementing standard GEMM for novelty.

7. Deep-Dive Questions

- Derive reuse within one K tile.** Each of $BM \times BK$ A elements is used across BN columns, and each of $BK \times BN$ B elements is used across BM rows. This amortizes global loads over $BM \times BN \times BK$ products.
- Why can split-K help and hurt?** Multiple programs split the K reduction to improve parallelism for small M/N, but their partial C tiles need atomics or a second reduction, adding traffic and possible non-determinism.

3. **What determines tensor-core use?** Supported input/accumulator dtypes, aligned/legal block dimensions, target hardware, and compiler lowering. Writing `tl.dot` is necessary in common kernels but not a universal performance guarantee.
4. **Why can transposed B be efficient or inefficient?** Efficiency depends on physical strides and which dimension adjacent lanes traverse. A logical transpose may make the K or N access contiguous; derive actual addresses rather than relying on the label.
5. **How does an epilogue affect numerical semantics?** Bias/activation before output casting can keep FP32 accuracy; casting before them changes rounding. Fusion must reproduce reference ordering, dtype promotion, and special-value behavior.

8. Comparison Tables

Naive scalar mapping		Blocked Triton GEMM
One output repeatedly loads a row and column		Tiles reuse loaded A/B values
Low effective reuse		High arithmetic intensity
Does not naturally express matrix hardware		<code>tl.dot</code> exposes block dot product
Simple indexing		More complex strides, masks, and tuning
Parameter	Increasing it may help	Increasing it may hurt
BM	Reuse B across more rows	Bigger accumulator/register pressure
BN	Reuse A across more columns	Bigger accumulator/register pressure
BK	Fewer loop iterations/pipeline opportunities	Larger operand tiles/resources
num_warps	More execution parallelism per program	Fewer resident programs/overhead
num_stages	Hides load latency through pipelining	More on-chip storage
Custom Triton matmul		Vendor library GEMM
Easy custom layouts/epilogues/research formats		Broad, heavily optimized standard coverage
Tunable source in Python		Mature heuristics and hardware-specific kernels
Maintenance/validation burden on you		Usually default choice for plain GEMM

9. Common Mistakes

- Forgetting that C accumulates across every K tile.
- Swapping B's K and N strides.
- Masking M/N but reading beyond K in the final iteration.
- Accumulating low-precision products in an unnecessarily low-precision accumulator.
- Selecting the largest tile without checking spills or occupancy.
- Assuming row-major layout despite arbitrary views/leading dimensions.
- Benchmarking only square, divisible matrices.
- Comparing a fused custom kernel against an unfused baseline unfairly.

10. Edge Cases / Special Cases

- **M**, **N**, or **K** equal to zero: define output and avoid invalid launches; **K=0** mathematically yields zeros before epilogue.
- Partial M/N/K tiles require different broadcast masks.
- Skinny or tiny matrices may be launch/parallelism limited; conventional square tiles can waste most work.
- Batched GEMM adds batch offsets/strides; broadcasting a batch operand may use a zero batch stride.
- Quantized/FP8/FP4 matmul needs scale layouts, accumulation rules, and hardware-specific constraints beyond ordinary FP16 GEMM.
- Split-K changes reduction order and may affect reproducibility.
- Very large address products should not overflow index arithmetic.

11. How to Explain in Interview

“I map each Triton program to a **BM×BN** tile of C. It loops over K in **BK** chunks, uses broadcasted stride-aware pointers to load A and B tiles with zero masks, calls **t1.dot**, and accumulates in FP32. Tile sizes trade data reuse against register pressure and occupancy, so I autotune a small candidate set and fuse only useful epilogue work.”

12. Quick Revision Notes

- Shapes: $(M \times K) @ (K \times N) = (M \times N)$.
- One program owns one C tile.
- K is the reduction/loop dimension.
- A mask: M and K; B mask: K and N; C mask: M and N.
- Invalid A/B values are zero.
- FP16/BF16 multiply + FP32 accumulate is common.
- Tune **BM**, **BN**, **BK**, warps, and stages.
- Trap: plain GEMM usually belongs to an optimized library.

13. Practice Tasks

1. Write a correct blocked kernel for arbitrary M/N/K and strides.
2. Manually trace **M=3**, **N=5**, **K=2** with **BM=2**, **BN=4**, **BK=2**.
3. Validate transposed A/B views and rectangular/tiny shapes.
4. Add bias and ReLU in the epilogue while keeping FP32 accumulation.
5. Autotune a small set of tiles and explain winners for square versus skinny matrices.
6. Calculate arithmetic intensity/reuse for two tile shapes.
7. Design a split-K variant on paper and identify where partial results are reduced.

14. Final Cheat Sheet

Item	Answer
Core definition	Tiled reduction computing C=A@B
Why it matters	Dominant compute primitive and best example of reuse/tuning
Most asked	Tile ownership, K loop, pointer strides, masks, FP32 accumulation

Item	Answer
Main comparison	Triton enables fused/custom GEMM; vendor libraries remain default for plain GEMM
One-line answer	"Own a C tile, stream K tiles of A/B, reuse them through <code>tl.dot</code> , accumulate in FP32, and mask every boundary."

LayerNorm

1. Overview

Definition. Layer normalization normalizes each row/features vector independently, then applies learnable scale `gamma` and bias `beta`:

$$y = ((x - \text{mean}) / \sqrt{\text{variance} + \text{eps}}) * \text{gamma} + \text{beta}.$$

For a row of `N` features, `mean` = $\sum x / N$ and `variance` = $\sum (x - \text{mean})^2 / N$. LayerNorm is central to transformers and other sequence models because it normalizes per example/token rather than across a batch. A Triton implementation demonstrates reductions, precision, saved statistics, backward formulas, fusion, and multi-stage gradient accumulation. Interviewers ask it because both correctness and memory behavior are subtle.

2. Core Idea

Think of grading scores within each student's own set of subjects: compute that student's average and spread, convert each score to a standardized value, then apply a learned per-subject scale and offset.

```
@triton.jit
def layernorm_fwd(x, y, gamma, beta, mean_out, rstd_out,
                  row_stride, N, eps,
                  BLOCK: tl.constexpr):
    row = tl.program_id(0)
    cols = tl.arange(0, BLOCK)
    mask = cols < N
    x_row = tl.load(x + row * row_stride + cols,
                    mask=mask, other=0.0).to(tl.float32)

    mean = tl.sum(x_row, axis=0) / N
    centered = tl.where(mask, x_row - mean, 0.0)
    var = tl.sum(centered * centered, axis=0) / N
    rstd = tl.rsqrt(var + eps)

    g = tl.load(gamma + cols, mask=mask).to(tl.float32)
    b = tl.load(beta + cols, mask=mask).to(tl.float32)
    out = centered * rstd * g + b
    tl.store(y + row * row_stride + cols, out, mask=mask)
    tl.store(mean_out + row, mean)
    tl.store(rstd_out + row, rstd)
```

One program normally owns one row when the feature dimension fits efficiently. Invalid lanes are zero for sums, but must be zeroed **after subtracting the mean**; otherwise padded lanes contain `-mean` and corrupt variance.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Normalization axis	Statistics are computed across the selected feature dimensions per row.	<code>[B,S,H]</code> LayerNorm over <code>H</code> .	It is not BatchNorm.
Mean reduction	Average of valid feature values.	<code>sum(x)/N</code> , not <code>/BLOCK</code> .	Padded lanes and divisor.
Variance	Common LayerNorm uses population variance over normalized features.	<code>sum((x-mean)^2)/N</code> .	Framework semantics and <code>unbiased=False</code> .
Epsilon	Stabilizes division when variance is zero/small.	<code>rsqrt(var+eps)</code> .	Exact placement affects semantics.
FP32 statistics	Lower-precision inputs are commonly upcast for reductions.	BF16 <code>x</code> , FP32 mean/variance.	Accuracy and overflow/cancellation.
Saved statistics	Forward stores mean and reciprocal standard deviation for backward.	One pair per row.	Memory-versus-recompute tradeoff.
Affine transform	Learned per-feature <code>gamma</code> and <code>beta</code> .	<code>xhat*gamma+beta</code> .	Their gradients reduce across rows.
Backward <code>dx</code>	Uses two row reductions involving <code>dy*gamma</code> .	Formula below.	Avoid materializing Jacobian.
<code>dgamma/dbeta</code>	Sum contributions from every row for each feature.	$\sum dy * xhat, \sum dy$.	Requires cross-program reduction strategy.
Large rows	A feature row may not fit efficiently in one program.	Large hidden/normalized shape.	Chunked/multi-pass fallback.

4. Real-World Example

In a pre-normalization transformer block, every token's hidden vector is normalized before attention or the MLP. For a tensor `[batch, sequence, hidden]`, flatten `batch*sequence` into rows and normalize `hidden` columns. A fused kernel may combine residual addition with LayerNorm, saving one read/write of the residual sum, while still saving the correct mean/rstd needed for training backward.

5. Diagrams / Mental Models

```

Input row x
|
+--> mean = sum(x)/N
|
+--> centered = x - mean
      |
      +--> variance = sum(centered2)/N
            |
            +--> rstd = 1/sqrt(variance+eps)
                  |
                  +-----+
                  |         |
gamma,beta -----+         v
                  |         |
                  y = centered*rstd*gamma + beta

```

Backward for one row, using $\text{xhat}=(x-\text{mean})*\text{rstd}$ and $\text{wdy}=\text{gamma}*dy$:

```

c1 = mean(xhat * wdy)
c2 = mean(wdy)
dx = rstd * (wdy - xhat*c1 - c2)
dgamma contribution = dy*xhat
dbeta contribution = dy

```

6. Common Interview Questions

Question	Clear answer	Key points expected	Common mistake
1. What does LayerNorm normalize?	Each sample/token across its normalized feature dimensions, independently of other batch elements.	Per-row statistics.	Describing BatchNorm batch statistics.
2. What is the forward formula?	$\text{xhat}=(x-\text{mean})/\text{sqrt}(\text{var}+\text{eps})$, then $y=\text{xhat}*\text{gamma}+\text{beta}$.	Mean, variance, epsilon, affine.	Omitting learned affine parameters.
3. Why add epsilon?	It prevents division by zero and limits amplification for tiny variance.	Numerical stability.	Adding epsilon after division or to rstd .
4. Why compute statistics in FP32?	Summation and squared differences are sensitive in FP16/BF16; FP32 lowers error/overflow risk.	Cast back only at output if required.	Believing FP32 removes all numerical differences.
5. Why divide by N rather than BLOCK?	Only N elements are logically valid; padded lanes are implementation details.	Correct mean/variance.	Counting masked padding.
6. Why mask centered	Padded loads are zero, but after $\text{x}-\text{mean}$ they would become $-\text{mean}$ and	$\text{where}(\text{mask}, \dots, 0)$.	Assuming a masked load remains harmless

Question	Clear answer	Key points expected	Common mistake
padding again?	pollute variance.		through subtraction.
7. What does forward save for backward?	Commonly mean and reciprocal standard deviation per row, sometimes other intermediates depending on memory/recompute policy.	Backward reuse.	Saving the full normalized tensor unconditionally.
8. What are $d\gamma$ and $d\beta$?	$d\gamma = \sum_{\text{rows}} dy * \hat{x}$; $d\beta = \sum_{\text{rows}} dy$, reduced per feature.	Cross-row reduction.	Returning per-row contributions as final gradients.
9. How is it different from RMSNorm?	RMSNorm divides by root mean square and usually does not subtract the mean; it often omits β .	Formula and cost difference.	Calling them equivalent.
10. Why fuse residual add with LayerNorm?	The residual sum can be consumed immediately for statistics/output instead of being written and reread.	Traffic saving with correct saved state.	Ignoring that backward may need the pre-normalized sum.

7. Deep-Dive Questions

- Derive the compact dx form.** Let $w_{dy} = dy * \gamma$ and $\hat{x} = (x - \text{mean}) * r_{std}$. Accounting for both mean and variance dependencies yields $dx = r_{std} * (w_{dy} - \text{mean}(w_{dy}) - \hat{x} * \text{mean}(w_{dy} * \hat{x}))$, avoiding an $N \times N$ Jacobian.
- Why might Welford's algorithm be used?** It maintains count, mean, and second central moment and can be more numerically robust/mergeable than separate naive sums. It costs more operations, so accuracy needs and target dtype determine whether it pays.
- How are affine gradients parallelized?** Each row program naturally produces feature-wise contributions, but many rows share each feature. Use atomics, grouped partial buffers followed by a second reduction, or a dedicated feature-wise reduction; choose based on contention and shape.
- Why save reciprocal standard deviation instead of variance?** Backward directly multiplies by r_{std} , so saving it avoids another square root/division; mean and r_{std} are only two scalars per row.
- How does epsilon placement affect backward compatibility?** $\text{sqrt}(\text{var} + \text{eps})$ is not the same as $\text{sqrt}(\text{var}) + \text{eps}$. Both forward values and gradients differ, especially for low variance, so match the framework definition exactly.

8. Comparison Tables

LayerNorm	BatchNorm	RMSNorm
Per sample/token across features	Per channel across batch/spatial samples	Per sample/token across features
Subtracts mean and divides by std	Uses batch mean/variance in training	Usually no mean subtraction

LayerNorm	BatchNorm	RMSNorm
Same training/inference statistic rule	Running statistics often used in inference	Same training/inference rule
Works well with variable/small batches	Quality depends on useful batch statistics	Slightly simpler reduction
Usually gamma and beta	Usually gamma and beta	Often gamma only, variant-dependent

Affine-gradient strategy	Advantage	Cost/risk
Direct atomics	Simple, no large partial buffer	Contention, dtype/support, nondeterministic order
Grouped partial buffers + reduce	Lower contention, scalable	Extra storage and second kernel
One feature program reduces rows	Simple ownership of final element	Access pattern/parallelism may be poor

Save stats	Recompute stats
Faster backward, tiny per-row storage	Lower saved-state memory
Forward writes mean/rstd	Backward rereads/reduces x
Common default	Useful under aggressive checkpointing/fusion

9. Common Mistakes

- Confusing LayerNorm with BatchNorm.
- Dividing by padded block width.
- Letting invalid lanes become `-mean` during variance.
- Using sample/unbiased variance instead of the framework's population variance.
- Computing reductions entirely in FP16/BF16.
- Placing epsilon outside the square root.
- Forgetting that `gamma/beta` gradients reduce across rows.
- Using one-row-per-program for unmanageably large feature dimensions without a fallback.

10. Edge Cases / Special Cases

- Constant row: variance is zero; epsilon makes the normalized pre-affine values zero rather than undefined.
- `N=1`: centered value is zero, so output is beta; gradients follow the exact formula/epsilon semantics.
- `N=0`: invalid normalization domain; reject in the wrapper.
- NaN/Inf input: usually propagates; tests should match framework behavior.
- Non-contiguous normalized dimensions require full stride support or a documented restriction.
- Very large `N` may require multiple passes/chunked Welford merging.
- Deterministic training may rule out unordered atomic accumulation for affine gradients.
- Mixed precision must match reference casting and output dtype.

11. How to Explain in Interview

"LayerNorm computes mean and population variance across each token's feature row, normalizes with $\text{rsqrt}(\text{var} + \text{eps})$, then applies per-feature gamma and beta. A Triton forward usually assigns one row per program, masks padding carefully, and computes statistics in FP32. Backward uses two row reductions for dx , while dgamma and dbeta require a reduction across rows."

12. Quick Revision Notes

- Normalize per row/features, not across batch.
- $\text{mean} = \text{sum}(x) / N$; $\text{var} = \text{sum}((x - \text{mean})^2) / N$.
- $\text{rstd} = \text{rsqrt}(\text{var} + \text{eps})$.
- Zero invalid lanes **after** centering.
- Save mean/rstd for backward.
- $\text{dx} = \text{rstd} * (\text{wdy} - \text{mean}(\text{wdy}) - \text{xhat} * \text{mean}(\text{wdy} * \text{xhat}))$.
- $\text{dgamma} = \sum \text{dy} * \text{xhat}$; $\text{dbeta} = \sum \text{dy}$ across rows.
- RMSNorm normally skips mean subtraction.

13. Practice Tasks

1. Implement forward LayerNorm and compare across FP32/FP16/BF16 inputs.
2. Test constant rows, $N=1$, all-negative values, and widths around powers of two.
3. Introduce the padded-centered bug deliberately and find a failing test.
4. Implement dx from the compact formula and compare with autograd.
5. Design partial-buffer reduction for $\text{dgamma}/\text{dbeta}$ and calculate storage.
6. Fuse residual addition with LayerNorm and list values needed by backward.
7. Implement RMSNorm by changing only the necessary statistics/affine logic.

14. Final Cheat Sheet

Item	Answer
Core definition	Per-row mean/variance normalization followed by learned affine transform
Why it matters	Transformer primitive combining reductions, numerical care, and fusion
Most asked	Axis, FP32 stats, padding, epsilon, backward reductions, RMSNorm
Main comparison	LayerNorm subtracts mean; RMSNorm usually does not; BatchNorm uses batch statistics
One-line answer	"Reduce each feature row in FP32, normalize with mean and rstd, apply gamma/beta, and reduce affine gradients across rows."

Fused Kernels

1. Overview

Definition. Kernel fusion combines multiple logically separate operations into one GPU kernel so intermediate values can remain in registers/on-chip storage instead of being written to and reread from global memory.

Fusion matters because many deep-learning operations are limited by memory bandwidth and launch overhead rather than arithmetic throughput. It is used for bias-activation epilogues, residual-add LayerNorm, softmax, dropout, optimizer updates, attention, and quantization pipelines. Interviewers ask when fusion helps, when it hurts, and how to preserve numerical/autograd semantics.

2. Core Idea

Without fusion, preparing a meal might mean placing chopped ingredients in the warehouse after every step and fetching them again for the next. Fusion keeps the intermediate on the workbench.

```
# Unfused logical pipeline:
# tmp1 = x + bias          # read x,bias; write tmp1
# tmp2 = relu(tmp1)       # read tmp1; write tmp2
# y = tmp2 * scale        # read tmp2,scale; write y

@triton.jit
def fused_bias_relu_scale(x, bias, scale, y, n,
                          BLOCK: tl.constexpr):
    offs = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    mask = offs < n
    v = tl.load(x + offs, mask=mask)
    b = tl.load(bias + offs, mask=mask)
    s = tl.load(scale + offs, mask=mask)
    out = tl.maximum(v + b, 0.0) * s
    tl.store(y + offs, out, mask=mask)
```

The fused kernel performs the same dataflow but removes the `tmp1` and `tmp2` global tensors and two launches. This is most valuable for elementwise/reduction chains with compatible ownership. Fusion does not automatically help if it creates huge live values, forces redundant computation, or blocks a superior library kernel.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Vertical/producer-consumer fusion	Consumer uses producer output immediately.	Bias → GELU.	Removes intermediate write/read.
Epilogue fusion	Adds cheap work to the output stage of a compute-heavy kernel.	GEMM + bias + activation.	Preserve high-performance core and avoid C reread.
Reduction fusion	Combines elementwise work around a reduction.	Mask + stable softmax.	Ownership and reduction scope must

Subtopic	Meaning and why it matters	Example	Common interview angle
			align.
Horizontal fusion	Combines independent small operations to reduce launches.	Update several small tensors.	May create awkward access divergence.
Memory traffic	Primary fusion benefit for bandwidth-bound chains.	Eliminate two accesses per intermediate.	Quantify bytes rather than just launches.
Launch overhead	Fewer kernels improve small/latency-sensitive workloads.	Inference serving microbatches.	Large kernels may care more about bandwidth.
Register pressure	More fused live values consume registers and can spill.	Attention keeps accumulators/statistics.	Over-fusion can be slower.
Recompute vs materialize	Recompute cheap values instead of saving large intermediates.	Recompute logits in attention backward.	Compare FLOPs with bytes/memory.
Semantics	Fused order, dtype, RNG, aliasing, and autograd must match contract.	Dropout mask reproducibility.	Speed does not excuse wrong behavior.
Fusion boundary	Stop where data ownership, synchronization, or resource costs become unfavorable.	Global reduction often needs another kernel.	Separate launches provide global synchronization.

4. Real-World Example

Flash-style attention is the canonical fused workload. A naive pipeline materializes QK^T , applies mask and softmax, then multiplies by V —an enormous score matrix is written and read several times. A fused tiled kernel computes score blocks, performs online softmax statistics, and accumulates $P@V$ without storing the full probability matrix. It saves memory capacity and bandwidth, although the implementation is considerably more constrained than a simple pointwise fusion.

5. Diagrams / Mental Models

Unfused:

X --K1--> TMP1 --global memory--K2--> TMP2 --global memory--K3--> Y

launch 1 launch 2 launch 3

Fused:

```
X --load--> [op1 -> op2 -> op3 kept program-local] --store--> Y
               one launch
```

Illustrative FP32 traffic per element for $y = \text{relu}(x + \text{bias}) * \text{scale}$:

Version	Reads/writes counted	Bytes/element
3 separate kernels	$x, \text{bias}, \text{tmp1}, \text{tmp1}, \text{tmp2}, \text{scale}, \text{tmp2}, y$	32
Fused kernel	$x, \text{bias}, \text{scale}, y$	16

The exact hardware traffic depends on caches and implementation, but the algorithmic intermediate traffic is halved.

6. Common Interview Questions

Question	Clear answer	Key points expected	Common mistake
1. What is kernel fusion?	Combining multiple operations into one kernel so intermediates remain local and launches/global traffic are reduced.	Traffic and launch reasoning.	Defining it merely as concatenating source code.
2. When does fusion help most?	Compatible bandwidth-bound producer/consumer chains with large intermediates or many small launches.	Profile and quantify bytes.	Saying it always helps.
3. Why can fusion hurt?	Higher register/on-chip pressure, spills, reduced occupancy, redundant computation, poor scheduling, or loss of optimized library dispatch.	Resource tradeoff.	Ignoring compiler/hardware limits.
4. What is epilogue fusion?	Applying bias/activation/scaling/quantization while a compute kernel's output tile is still local, before final store.	Common GEMM pattern.	Calling a preceding unrelated reduction an epilogue.
5. Can kernels communicate globally inside a fusion?	Program instances lack a general cheap global barrier; dependent global phases may still require separate launches or specialized algorithms.	Launch boundary as synchronization.	Assuming one kernel can freely synchronize the entire grid.
6. How do you estimate benefit?	Compare eliminated intermediate bytes and launches against added FLOPs/resources; validate with profiling.	Roofline and measured performance.	Counting only source lines.
7. How does fusion affect autograd?	Backward must reproduce correct gradients and decide which intermediates/statistics to save or recompute.	Forward speed can shift cost to backward.	Implementing forward only in a training path.

Question	Clear answer	Key points expected	Common mistake
8. What about numerical equivalence?	Changed operation order/casts can alter floating-point results; match documented tolerances and semantic order.	Non-associativity and dtype.	Demanding bitwise identity or ignoring large error.
9. How does dropout complicate fusion?	RNG must be counter-based/deterministic under the framework contract, and backward needs the same mask or reproducible seed/offset.	Random-state semantics.	Calling a stateful random function per lane without mapping guarantees.
10. Where should fusion stop?	At a boundary where data must be globally shared/synchronized, tile ownership changes badly, resources spill, or an existing optimized kernel should remain intact.	Evidence-driven boundary.	Maximally fusing the whole model.

7. Deep-Dive Questions

- How does fusion change arithmetic intensity?** Eliminating intermediate bytes while retaining or modestly increasing FLOPs raises useful work per global byte, potentially moving a kernel away from the bandwidth roof toward compute limits.
- Why is recomputation sometimes optimal?** A few extra arithmetic operations can be cheaper than writing, storing, and rereading a large tensor. Attention backward and activation checkpointing use this compute-for-memory trade.
- What prevents fusion across a global reduction?** A later phase may require all programs' partial results. Without a grid-wide barrier and safe visibility, use a second kernel or redesign with hierarchical/persistent methods.
- How would you diagnose over-fusion?** Compare variants and inspect register count, occupancy, local-memory spills, achieved bandwidth/compute, and time per shape. A faster individual stage can still make the fused whole slower.
- How do fusion and compilation interact?** Many optional operations/dtypes/shapes can create numerous specialized variants and long compile times. Keep meta-parameter axes tied to real workload differences and cache compiled variants.

8. Comparison Tables

Unfused kernels	Fused kernel
Modular operators and easy reuse	Specialized combined dataflow
More launches	Fewer launches
Intermediate tensors materialized	Intermediates often program-local
Natural global synchronization between launches	No general grid-wide barrier inside
Lower per-kernel resource pressure	More live state/register pressure
Easier independent tuning/debugging	End-to-end tuning required

Fusion type	Example	Primary benefit	Main risk
Pointwise vertical	bias + activation + scale	Remove intermediates	Modest; usually easiest
Epilogue	matmul + bias + GELU	Avoid reading/writing C again	Disturbing tuned GEMM/resource use
Reduction	masked softmax	Collapse many passes	Large reduction block/resources
Complex tiled	fused attention	Avoid $S \times S$ materialization	Algorithmic and numerical complexity
Horizontal	several tiny updates	Reduce launch overhead	Divergent layouts/work imbalance
Save intermediate		Recompute intermediate	
More memory and bandwidth		More FLOPs	
Faster if recomputation is expensive		Faster if value is cheap and memory-bound	
Easier debugging		More complex backward/dataflow	

9. Common Mistakes

- Assuming fewer kernel launches always means lower runtime.
- Fusing across a dependency that requires global synchronization.
- Ignoring register spills and occupancy loss.
- Changing dtype/cast/order and silently breaking numerical semantics.
- Losing an optimized vendor GEMM while chasing fusion.
- Forgetting training backward, saved tensors, or RNG reproducibility.
- Comparing fused and unfused versions with different mathematical work.
- Writing one giant kernel for many unrelated shapes and branches.

10. Edge Cases / Special Cases

- Aliased inputs/outputs can change correctness because fusion changes read/write timing.
- Empty/small inputs may need host-side handling; launch reduction is especially valuable for tiny work.
- All-masked reductions still require explicit defined behavior inside a fused kernel.
- Dynamic shapes can multiply compiled variants or select a poorly tuned generic configuration.
- Very large tiles may spill intermediates to local/global memory, erasing the traffic benefit.
- Random operations require stable element-to-counter mapping across forward/backward and launch configurations.
- Deterministic reduction requirements may restrict atomics or traversal changes.
- Debugging should compare intermediate logical stages against an unfused reference, even if production never materializes them.

11. How to Explain in Interview

“Kernel fusion combines producer-consumer operations so intermediate tensors stay in registers or on-chip storage. Its main wins are fewer global-memory round trips and launches, which is ideal for bandwidth-bound

chains such as bias plus activation or softmax. I stop fusing when it requires global synchronization, causes register spills or occupancy loss, changes semantics, or replaces a better optimized library kernel."

12. Quick Revision Notes

- Fusion saves intermediate reads/writes and launch overhead.
- Best for compatible bandwidth-bound chains.
- Epilogue fusion: add cheap work before a compute kernel stores output.
- Over-fusion: registers, spills, occupancy, branches, compilation variants.
- Kernel boundary often supplies global synchronization.
- Preserve dtype, operation order, alias, RNG, and autograd behavior.
- Measure bytes saved and end-to-end time.

13. Practice Tasks

1. Fuse vector add + ReLU and calculate algorithmic bytes saved.
2. Implement bias + activation as a matmul epilogue and compare against separate calls.
3. Profile register use and runtime as one more operation is added to a fused kernel.
4. Write an unfused reference for residual + LayerNorm, then fuse it and validate saved statistics/backward requirements.
5. Explain why two-pass global reduction cannot always become one ordinary fused kernel.
6. Design deterministic counter mapping for fused dropout conceptually.
7. Compare save-versus-recompute options for softmax probabilities in training.

14. Final Cheat Sheet

Item	Answer
Core definition	Combine operations so intermediates remain local within one kernel
Why it matters	Cuts global traffic, temporary memory, and launch overhead
Most asked	When fusion helps/hurts, epilogues, global barriers, registers, autograd
Main comparison	Unfused is modular; fused is traffic-efficient but specialized/resource-heavy
One-line answer	"Fuse compatible producer-consumer work until memory traffic falls without breaking semantics or spilling the saved state back out."

Cross-Topic Interview Map

If the interviewer asks...	Connect these concepts
"How is work assigned?"	Launch grid → program IDs → blocks → pointer offsets
"How do arbitrary shapes work?"	Ceiling grid → padded block → masked load/store → correct identity
"Why is this kernel fast?"	Coalescing + reuse + arithmetic intensity + fewer global-memory passes

If the interviewer asks...	Connect these concepts
"Why is it numerically stable?"	FP32 accumulation + max subtraction/epsilon + reduction identities
"What happens for a huge dimension?"	One-program resource limit → tiling/online reduction/multi-pass fallback
"Should we fuse it?"	Bytes and launches saved versus registers, synchronization, semantics, and library quality

Recommended Study Order

1. Implement vector addition until grid, IDs, offsets, and masks are automatic.
2. Implement row sum/max to learn blocks and reduction identities.
3. Implement stable fused softmax and test adversarial values.
4. Implement LayerNorm forward, then derive backward on paper.
5. Trace tiled matrix multiplication with tiny shapes before tuning it.
6. Fuse only after profiling an unfused correct reference.

Official References

- [Triton documentation](#)
- [Vector Addition tutorial](#)
- [Fused Softmax tutorial](#)
- [Matrix Multiplication tutorial](#)
- [Layer Normalization tutorial](#)
- [Fused Attention tutorial](#)